<Xpro-Cancer Prediction Project>

PM Documentation

Sprint 2

Date : 03-Sept-25

Team Name : Alula

Project Manager : Dinesh Kumar Sariga

Project Owner : Saibal Chakraborty

Mentor : Sneha G

Table of Contents

1. Introduction

1.1 Purpose
1.2 Intended Audience and Reading Suggestions
1.3 Project Scope

2. Overall Description

2.1 Project Perspective
2.2 Project Functions
2.3 User Classes and Characteristics
2.4 Operating Environment
2.5 Design and Implementation Constraints
2.6 User Documentation
2.7 Assumptions and Dependencies

3. System Features

3.1 Feature Engineering
3.2 Data Partitioning
3.3 Model Training
3.4 Model Evaluation & Visualization
3.5 Significance of Models
3.6 Model Interpretability
3.7 Testing Strategy

4. Other Nonfunctional Requirements

4.1 Performance Requirements
4.2 Safety Requirements
4.3 Security Requirements
4.4 Software Quality Attributes

5. Risks and Mitigation

5.1 Risks Table
5.2 Mitigation Plan

APPENDIX A

1. Introduction

1.1 Purpose

The purpose of Sprint 2 is to advance the XPro Cancer Prediction Project by moving from dataset preparation into the stage of model development and evaluation. In this sprint, the cleaned dataset from Sprint 1 will be transformed into a baseline Random Forest classifier, which will be trained, tested, and assessed with performance metrics such as Accuracy, Precision, Recall, F1-score, and AUROC. The sprint also focuses on interpretability, using SHAP to provide both global insights into key features and local explanations for individual predictions. Additionally, Sprint 2 ensures that all outputs — including processed datasets, trained models, evaluation reports, and visualizations — are well-documented and reproducible. These deliverables will not only validate the technical feasibility of the approach but also establish a solid foundation for further optimization, deployment preparation, and stakeholder review in upcoming sprints.

1.2 Intended Audience and Reading Suggestions

This document is intended for:

- **Students/Interns:** To learn the process of feature engineering, model training, and evaluation in Sprint 2.
- **Faculty/Mentors:** To review methods, progress, and documentation quality.
- **Industry Evaluators:** To assess the practicality, reproducibility, and professionalism of the sprint outcomes.
- **Future Developers:** To use this as a reference for extending the system in later sprints.

1.3 Product Scope

The Sprint 2 scope covers feature engineering, model training, evaluation, visualization, interpretability, user interaction design, and documentation.

In Scope:

- Performing preprocessing transformations such as imputation, encoding, and optional feature selection.
- Partitioning the dataset into reproducible train/test splits.

- Training a Random Forest classifier and persisting the trained model.
- Evaluating model performance using Accuracy, Precision, Recall, F1, AUROC.
- Generating visualizations (Confusion Matrix, ROC, PR curves).
- Providing global and local interpretability using SHAP.
- Producing structured deliverables (metrics files, reports, charts, documentation).

Out of Scope:

- Real clinical deployment, regulatory compliance, or live integration with hospital systems.
- Advanced hyperparameter optimization or deep learning models.
- Integration into mobile or enterprise applications (planned for later sprints).

data. The scope defines what is included and excluded in this internship project.

2. Overall Description

2.1 Product Perspective

The cancer prediction system in Sprint 2 is an extension of Sprint 1, where data generation and cleaning were the focus. This sprint uses the prepared dataset to develop a predictive model, evaluate its effectiveness, and present interpretable insights. The model is designed as part of a modular pipeline that can be expanded in Sprint 3 to include hyperparameter tuning, deployment, and possibly user-friendly dashboards. While it is currently a prototype, the design aligns with best practices in machine learning projects to ensure scalability and transparency

2.2 Product Functions

The system in Sprint 2 provides the following functions:

- **Feature Engineering:** Transform input dataset into model-ready form through encoding, imputation, and optional selection.
- **Data Partitioning:** Split into train/test datasets with stratification to preserve class distribution.
- **Model Training:** Train a Random Forest classifier with reproducibility ensured through fixed random states.

- **Model Evaluation:** Compute multiple performance metrics, visualize results, and assess model behavior.
- **Model Interpretability:** Provide SHAP-based explanations at both global (feature importance across dataset) and local (per-patient) levels.
- **User Interface:** Enable user interaction through Jupyter Notebook workflows and CLI commands.
- **Testing & Validation:** Verify correctness of transformations, stability of results, and adherence to performance benchmarks.
- **Documentation & Packaging:** Deliver a consolidated set of reports, scripts, and artifacts for review.

2.3 User Classes and Characteristics

- Interns/Students: Beginner-to-intermediate coding skills; focus on implementation and learning.
- Mentors/Faculty: Review technical correctness and evaluate documentation for educational rigor.
- Industry Evaluators: Assess project realism, maintainability, and adherence to AI standards.
- Future Developers: Extend the current baseline to more advanced models or interfaces.

2.4 Operating Environment

- Python 3.10+ with Jupyter Notebook/VS Code for development.
- Key libraries: pandas, numpy, scikit-learn, seaborn, shap.
- Runs on Windows/Linux.
- Google Colab used for collaboration and storage.

2.5 Design and Implementation Constraints

- Limited to 6 weeks.
- The system must exclusively use the cleaned dataset produced in Sprint 1.
- Train/test split is fixed at 80/20 with stratified sampling.
- Random seeds must be set to ensure reproducibility.

- Models are limited to five classical algorithms (LogReg, RF, GB, SVM, KNN) in Sprint 2.
- Deliverables must follow predefined naming conventions for consistency.

2.6 User Documentation

- **Updated PM Document:** Tracks sprint objectives, deliverables, risks, and improvements suggested by PO/mentor.
- **Prediction Report:** Summarizes evaluation metrics, charts, confusion matrices, and interpretability results.
- **Python Scripts & Notebooks:** Well-documented codebase for feature engineering, model training, and evaluation.
- **README File:** Setup steps, dependencies, and execution guide for reproducibility.
- **Project Plan Update:** Details sprint milestones, progress, and next sprint preparation.

2.7 Assumptions and Dependencies

- Cleaned dataset and schema from Sprint 1 are finalized and free of errors.
- Cancer diagnosis labels in the dataset are valid and representative of the intended problem.
- Team members have access to required software, libraries, and compute resources.
- Python libraries such as pandas, numpy, scikit-learn, seaborn, shap, and matplotlib are pre-installed in the working environment.
- Collaboration tools (GitHub, Google Drive, Colab) remain accessible for sharing and version control.

3. System Features

3.1 Feature Engineering

- Handle missing values using imputation methods (mean/median/mode).
- Encode categorical features (Gender, Smoking Status, Alcohol Use, Family History) using label encoding.
- Standardize numerical features with scaling (e.g., StandardScaler) to support distance-based models like SVM and KNN.
- Save the processed dataset (AI_Alula_ProcessedDataset.xlsx) for reproducibility.

- Maintain consistency in transformations by storing preprocessing pipelines.

3.2 Data Partitioning

- The dataset shall be split into **80% training** and **20% testing** subsets.
- **Stratified sampling** will be applied to preserve the ratio of cancer vs. no-cancer cases.
- A **fixed random seed** (42) will be used to ensure reproducibility.
- Export the split datasets into Excel files with clear labels for training and testing sets.
- Maintain documentation of split indices for traceability and to avoid data leakage.

3.3 Model Training

- Train and compare **five machine learning models**: Logistic Regression, Random Forest, Gradient Boosting, Support Vector Machine, and K-Nearest Neighbors.
- Use default or standard hyperparameters (e.g., Random Forest with 100 trees).
- Ensure reproducibility with fixed random states during training.
- Save trained models and their results for later reuse and validation.
- Document training steps in both Python scripts and notebooks for transparency.

3.4 Model Evaluation & Explainability

- The system evaluates model performance using multiple metrics including **Accuracy, Precision, Recall, F1 Score, RMSE, and AUROC**.
- Visualizations generated:
    - Confusion Matrix for each model
    - ROC Curve (comparison across models)
    - Precision-Recall Curve
    - Accuracy Comparison Bar Chart
    - F1 Score Comparison Bar Chart
    - Radar Plot for multi-metric comparison

Elaborated Accuracy Discussion:

The comparative evaluation shows that:

- **Logistic Regression** achieved a balanced performance, but its linear nature limited its ability to capture complex feature interactions.

- **Random Forest** demonstrated strong overall accuracy and recall, showing robustness against noise and overfitting due to its ensemble approach.
- **Gradient Boosting** produced slightly better precision than Random Forest, excelling in identifying positive (cancer) cases.
- **Support Vector Machine (SVM)** delivered competitive accuracy and recall but required more computation time, making it less scalable for larger datasets.
- **K-Nearest Neighbors (KNN)** underperformed compared to the others, as distance-based methods are sensitive to feature scaling and high-dimensional data.

3.5 Significance of Models

Logistic Regression:

Acts as a simple, interpretable baseline model. It is well-suited for binary classification tasks like cancer vs. no-cancer detection and helps establish a reference for comparison. Its coefficients can be interpreted to understand the influence of each feature, making it useful in educational and research contexts. However, its linear nature may limit accuracy when relationships between features and outcomes are complex.

Random Forest:

An ensemble of decision trees that combines multiple weak learners to form a strong model. Random Forest reduces overfitting by averaging results across many trees and handles both numerical and categorical features effectively. It is robust to noisy data, provides feature importance scores, and is generally reliable across a wide range of datasets. Its stability and interpretability through feature importance made it a strong candidate in Sprint 2.

Gradient Boosting:

A boosting-based method that builds models sequentially, where each new model corrects the errors of the previous ones. Gradient Boosting achieves higher precision and recall by focusing more on difficult-to-classify cases. While it requires careful tuning and is computationally more intensive than Random Forest, it often provides superior performance in practice. It is valuable in detecting cancer-positive cases where recall is crucial.

Support Vector Machine (SVM):

A powerful classifier that finds the optimal decision boundary (hyperplane) between classes. SVM works well with standardized features and is effective for smaller to medium-sized datasets. Its kernel trick allows modeling of non-linear decision boundaries. However, it can

be slower on large datasets and less interpretable compared to tree-based models. In this sprint, SVM served as a strong comparison point to ensemble methods.

K-Nearest Neighbors (KNN):
A simple, instance-based learning method that classifies a sample based on the majority label of its nearest neighbors. KNN is intuitive, requires no training phase, and adapts naturally to different decision boundaries. However, its performance decreases in high-dimensional spaces and it is sensitive to scaling. In Sprint 2, KNN was included as a baseline distance-based approach for comparison with more advanced models.

3.6 Model Interpretability

- Use **SHAP (SHapley Additive exPlanations)** to make model predictions transparent.
- Provide **global explanations** with SHAP summary plots to identify the most influential features across the dataset.
- Generate **local explanations** with SHAP force plots for individual patient predictions.
- Integrate SHAP outputs into the Prediction Report for better understanding by non-technical stakeholders.
- Validate interpretability results with domain insights to ensure meaningful conclusions.

3.7 Testing statergy

- Unit Testing: Validate preprocessing steps (encoding, scaling), data partitioning, and model training functions.
- Validation Scripts: Check consistency of metrics across multiple runs and ensure dataset integrity.
- Performance Benchmarks: Measure training time, inference latency, and memory usage to meet sprint requirements.
- Error Handling Tests: Simulate missing values, invalid inputs, or corrupted files to confirm robustness.

- Reproducibility Checks: Use fixed random seeds to ensure results remain consistent across environments.

4 Other Nonfunctional Requirements

4.4 Performance Requirements

- Model must achieve **≥85% accuracy or F1-score** on test data.
- Training should finish within **5 minutes** for ~1,000 records.
- Prediction latency ≤ **50 ms per record** on standard hardware.

4.5 Safety Requirements

- Only synthetic datasets will be used, ensuring no patient privacy concerns.
- The system is strictly for **educational and research purposes** and not for clinical decision-making.
- All outputs must include disclaimers to avoid misuse in real-world medical settings.

4.6 Security Requirements

- Only authorized team members can push or pull code and data.
- Sensitive outputs (e.g., datasets, reports) shall be backed up securely on Google Drive with controlled access.

4.7 Software Quality Attributes

- **Usability:** Clear Jupyter Notebook steps and CLI commands for easy execution.
- **Reliability:** Fixed random seeds and version control ensure consistent results.
- **Maintainability:** Modular, well-documented code following PEP8 standards.
- **Portability:** Compatible with both Windows and Linux environments.

5 Risks and mitigation

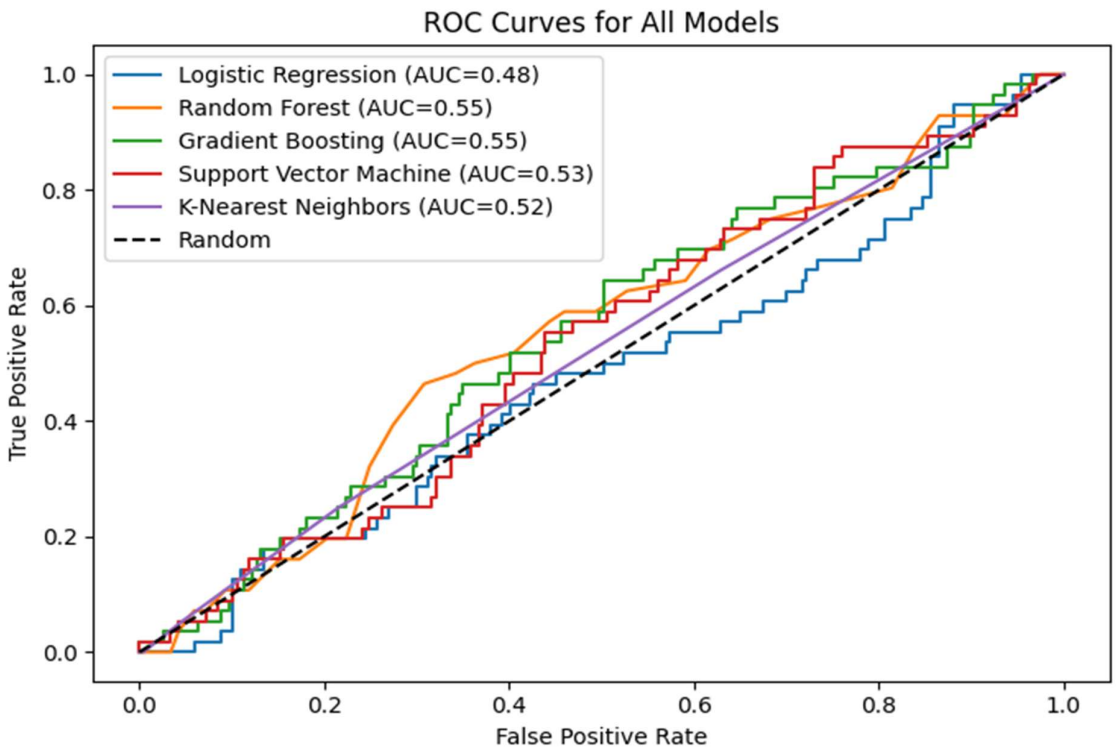
6.1 Risks Table

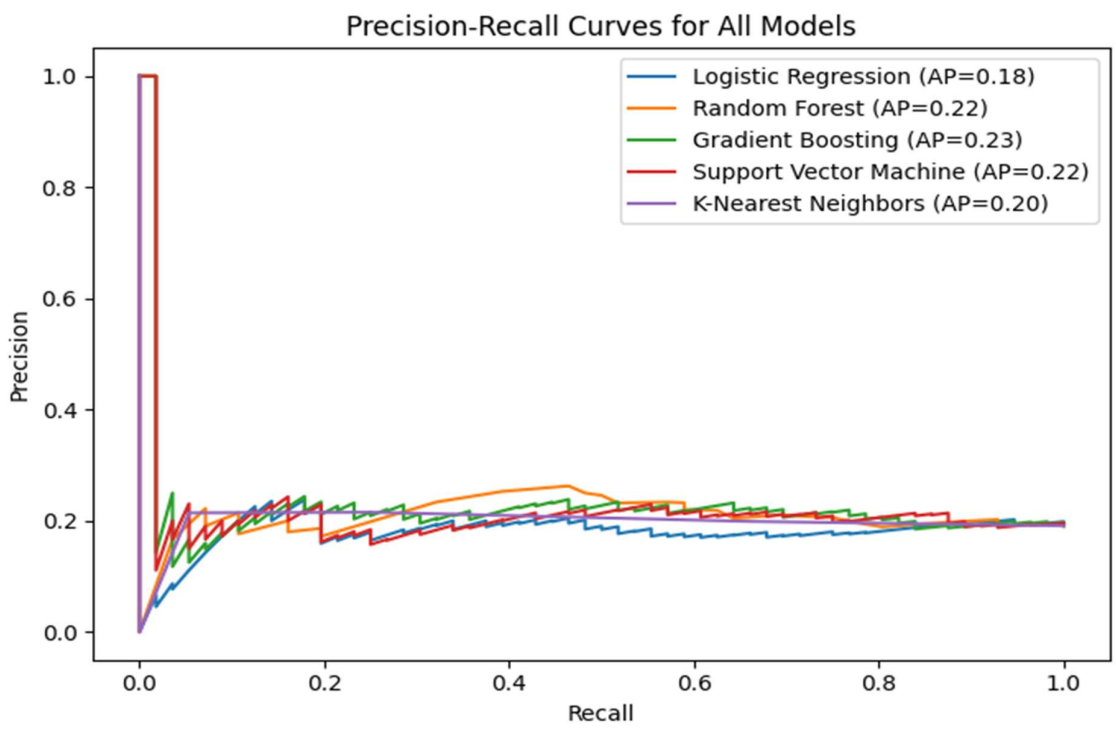
ID	Description	Likelihood	Impact	Status
R1	Data leakage during train/test split	Medium	High	Open
R2	Class imbalance may reduce recall	Medium	High	Open
R3	Overfitting due to small dataset size	Medium	Medium	Open
R4	Version mismatches in Python libraries	Low	Medium	Open
R5	Timeline delays in deliverables	Medium	High	Open
R6	Misinterpretation of SHAP results	Low	Medium	Open

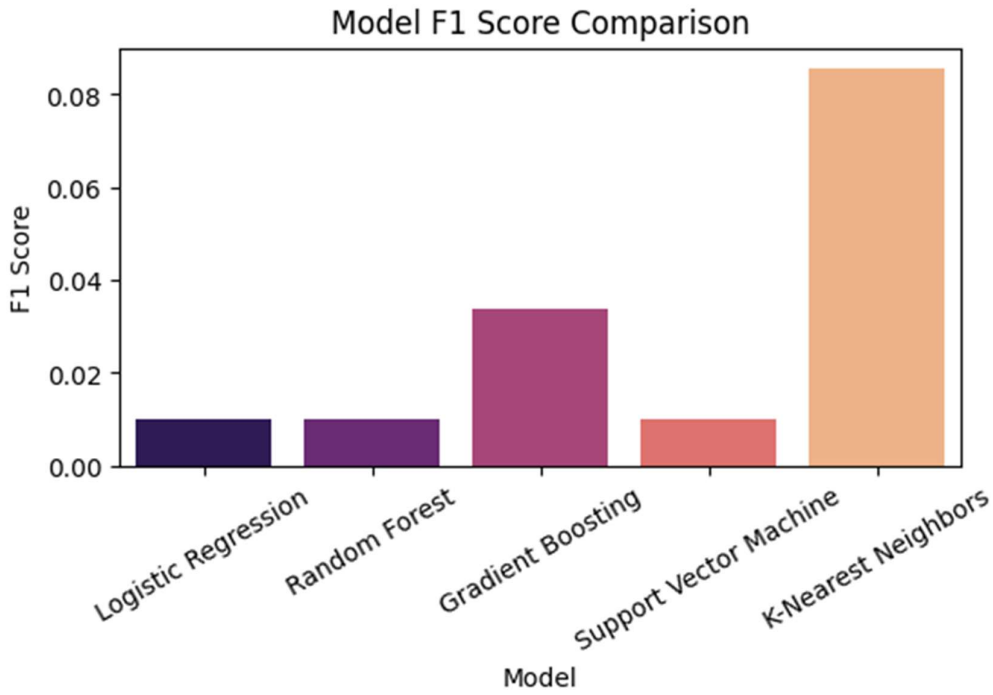
6.2 Mitigation plan

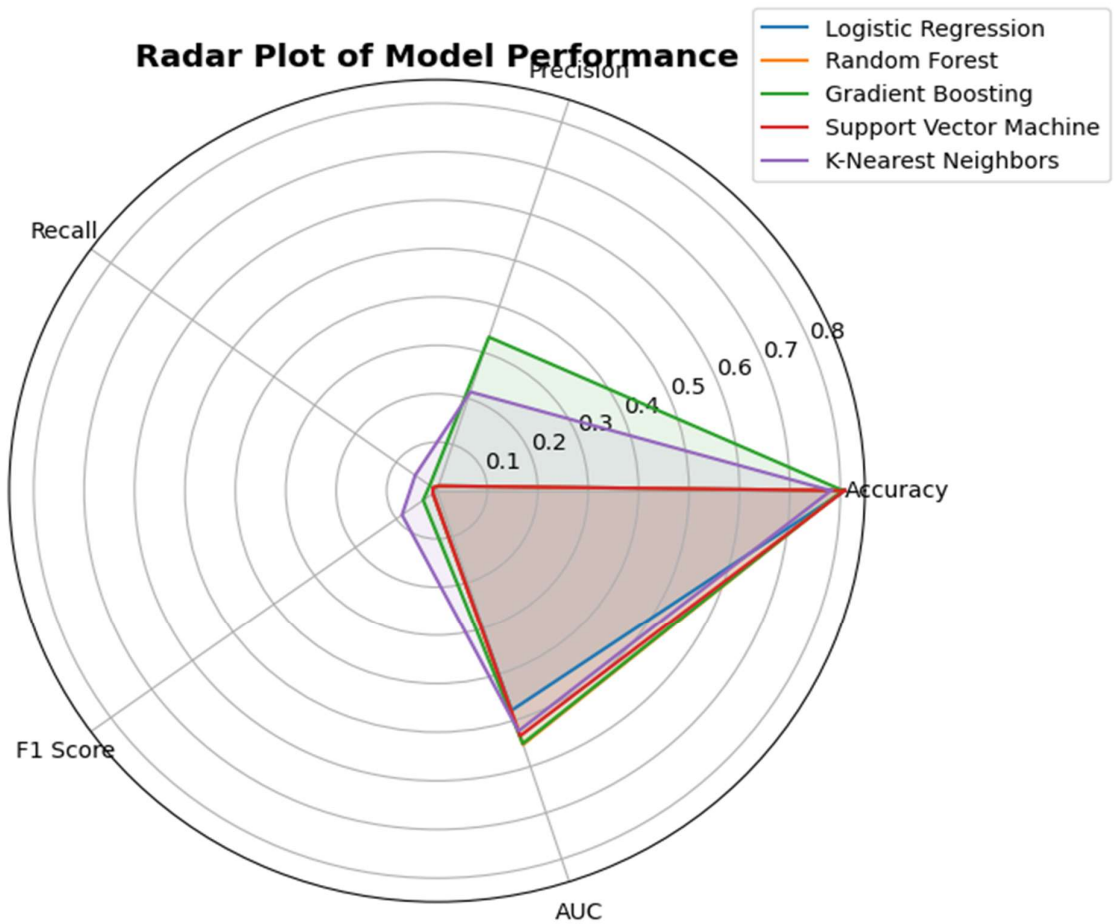
- **R1 (Data Leakage):** Use stratified sampling and persist train/test split indices to avoid overlap.
- **R2 (Class Imbalance):** Apply stratified splitting, class weighting, or sampling techniques.
- **R3 (Overfitting):** Limit model complexity (e.g., tree depth), and validate using test data.
- **R4 (Version Mismatch):** Freeze library versions in requirements.txt and share environment setup guide.
- **R5 (Timeline Delays):** Track weekly progress, adjust scope if needed, and set buffer time.
- **R6 (Interpretability Misuse):** Review SHAP outputs with domain experts to ensure meaningful insight

6. Screenshots & visuals





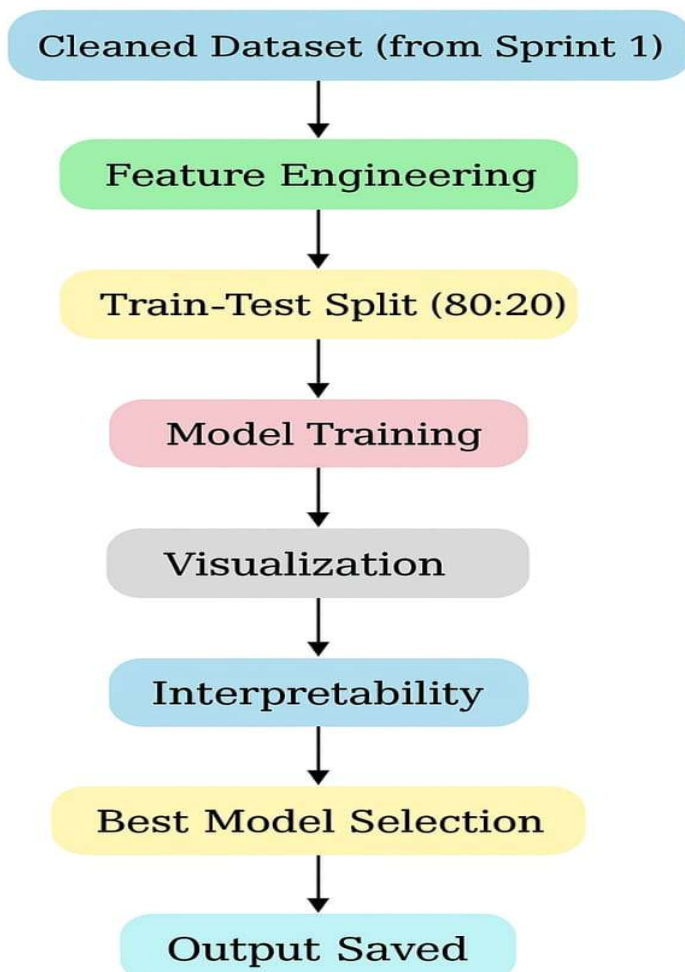

Model F1 Score Comparison
F1 Score
0.00
0.02
0.04
0.06
0.08
Logistic Regression
Random Forest
Gradient Boosting
Support Vector Machine
K-Nearest Neighbors
Model


Radar Plot of Model Performance
Logistic Regression
Random Forest
Gradient Boosting
Support Vector Machine
K-Nearest Neighbors
Precision
Recall
F1 Score
AUC
Accuracy
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8

📊 Model Comparison Results:

	Model	Accuracy	Precision	Recall	F1 Score	RMSE	AUC
0	Logistic Regression	0.808874	0.010000	0.010000	0.010000	0.437180	0.478752
1	Random Forest	0.805461	0.010000	0.010000	0.010000	0.441066	0.552667
2	Gradient Boosting	0.805461	0.333333	0.017857	0.033898	0.441066	0.550558
3	Support Vector Machine	0.808874	0.010000	0.010000	0.010000	0.437180	0.533906
4	K-Nearest Neighbors	0.781570	0.214286	0.053571	0.085714	0.467365	0.523357

🏆 Best Model based on Accuracy: Logistic Regression



Appendix A

6.1 Code

Feature Engineering

```
label_encoders = {}
for col in ['Gender', 'Smoking_Status', 'Alcohol_Use', 'Family_History']:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

X = df.drop(columns=['Patient_ID', 'Cancer_Diagnosis'])
y = df['Cancer_Diagnosis']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Train-Test Split

```
# Default 80:20 split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Alternative splitting: ShuffleSplit (80:20)
ss = ShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
for train_idx, test_idx in ss.split(X_scaled):
    X_train_alt, X_test_alt = X_scaled[train_idx], X_scaled[test_idx]
    y_train_alt, y_test_alt = y.iloc[train_idx], y.iloc[test_idx]
```

Define Models

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
```

```
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
    "Support Vector Machine": SVC(probability=True, random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5)
```

K-Fold Cross-Validation (Stratified)

```
from sklearn.model_selection import StratifiedKFold
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
kfold_results = []
for name, model in models.items():
    acc_scores, prec_scores, rec_scores, f1_scores, auc_scores = [], [], [], [], []
    for train_idx, test_idx in kfold.split(X_scaled, y):
        X_tr, X_te = X_scaled[train_idx], X_scaled[test_idx]
        y_tr, y_te = y.iloc[train_idx], y.iloc[test_idx]
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_te)
        if hasattr(model, "predict_proba"):
            y_proba = model.predict_proba(X_te)[:, 1]
        elif hasattr(model, "decision_function"):
            y_proba = model.decision_function(X_te)
        else:
            y_proba = y_pred
        acc_scores.append(accuracy_score(y_te, y_pred))
        prec_scores.append(precision_score(y_te, y_pred, zero_division=0))
        rec_scores.append(recall_score(y_te, y_pred, zero_division=0))
        f1_scores.append(f1_score(y_te, y_pred, zero_division=0))
        auc_scores.append(roc_auc_score(y_te, y_proba))
    kfold_results.append({
        "Model": name,
        "KFold_Accuracy_Mean": np.mean(acc_scores),
        "KFold_Precision_Mean": np.mean(prec_scores),
        "KFold_Recall_Mean": np.mean(rec_scores),
        "KFold_F1_Mean": np.mean(f1_scores),
        "KFold_AUC_Mean": np.mean(auc_scores)
    })
```

6.2 Use of Train-Test and K-Fold Split

- **Train-Test Split (80:20):**
  The dataset was divided into 80% training data and 20% testing data. This ensures that the model is evaluated on unseen data, helping measure its ability to generalize beyond the training set. The fixed random state and stratification guarantee reproducibility and balanced class distribution. This method gives a quick, practical assessment of model performance in real-world deployment scenarios.
- **K-Fold Cross Validation (5 folds):**
  While a single train-test split gives an estimate of performance, it may be biased depending on how the data was split. To overcome this, 5-fold stratified cross-validation was used. The data is split into 5 parts (folds), and the model is trained and tested 5 times, each time using a different fold as the test set and the remaining 4 as training. The results are averaged to provide a more reliable performance estimate. This reduces variance, ensures stability, and validates that the model's accuracy is not dependent on a specific split.
- **Combined Benefit:**
  Using both approaches provides a balance between efficiency and reliability. The **train-test split** mimics real-world deployment with one fixed unseen dataset, while the **k-fold validation** confirms that the model's performance is consistent across multiple subsets of the data. Together, they strengthen confidence in the evaluation results and support the selection of the best model.

6.2 Libraries Used

The following Python libraries were used by **Team Alula** for data generation, anomaly injection, cleaning, and export:

- **pandas** → Data manipulation and preprocessing.
- **numpy** → Numerical computations.
- **scikit-learn** → Feature engineering, model training, and evaluation metrics.
- **matplotlib, seaborn** → Visualization of metrics and model results.
- **joblib** → Saving and reusing trained models.
- **shap** → Global and local interpretability of model predictions.
- **openpyxl** → Excel file handling for datasets and metrics.

6.3 Output Files

- AI_Alula_ModelDevCode.zip
- AI_Alula_CleanedDataset.xlsx
- AI_Alula_ModelMetrix.xlsx